Intuitive Understanding of Kalman Filtering with MATLAB®

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Intuitive Understanding of Kalman Filtering with MATLAB®

Armando Barreto

Malek Adjouadi

Francisco R. Ortega

Nonnarit O-larnnithipong

CRC Press
Taylor & Francis Group
Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

MATLAB® is a trademark of The MathWorks, Inc. and is used with permission. The MathWorks does not warrant the accuracy of the text or exercises in this book. This book's use or discussion of MATLAB® software or related products does not constitute endorsement or sponsorship by The MathWorks of a particular pedagogical approach or particular use of the MATLAB® software.

First edition published 2021
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487–2742

and by CRC Press
2 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

© 2021 Taylor & Francis Group, LLC

CRC Press is an imprint of Taylor & Francis Group, LLC

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978–750–8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data
A catalog record for this book has been requested

ISBN: 978-0-367-19135-1 (hbk)
ISBN: 978-0-367-19133-7 (pbk)
ISBN: 978-0-429-20065-6 (ebk)

Typeset in Minion Pro
by Apex CoVantage, LLC

Contents

Acknowledgments, xi

Authors, xiii

Introduction, xv

PART I **Background**

CHAPTER 1 ■ System Models and Random Variables 3

1.1 DETERMINISTIC AND RANDOM MODELS AND VARIABLES 3

1.2 HISTOGRAMS AND PROBABILITY FUNCTIONS 6

1.3 THE GAUSSIAN (NORMAL) DISTRIBUTION 12

1.4 MODIFICATION OF A SIGNAL WITH GAUSSIAN DISTRIBUTION THROUGH A FUNCTION REPRESENTED BY A STRAIGHT LINE 14

1.5 EFFECTS OF MULTIPLYING TWO GAUSSIAN DISTRIBUTIONS 21

CHAPTER 2 ■ Multiple Random Sequences Considered Jointly 25

2.1 JOINT DISTRIBUTIONS—BIVARIATE CASE 25

2.2 BIVARIATE GAUSSIAN DISTRIBUTION—COVARIANCE AND CORRELATION 32

2.3 COVARIANCE MATRIX 38

2.4 PROCESSING A MULTIDIMENSIONAL GAUSSIAN DISTRIBUTION THROUGH A LINEAR TRANSFORMATION 39

2.5 MULTIPLYING TWO MULTIVARIATE GAUSSIAN DISTRIBUTIONS 40

CHAPTER 3 ■ Conditional Probability, Bayes' Rule and Bayesian Estimation 45

3.1 CONDITIONAL PROBABILITY AND THE BAYES' RULE 45

3.2 BAYES' RULE FOR DISTRIBUTIONS 50

PART II **Where Does Kalman Filtering Apply and What Does It Intend to Do?**

CHAPTER 4 ■ A Simple Scenario Where Kalman Filtering May Be Applied 55

4.1 A SIMPLE MODELING SCENARIO: DC MOTOR CONNECTED TO A CAR BATTERY 55

4.2 POSSIBILITY TO ESTIMATE THE STATE VARIABLE BY PREDICTION FROM THE MODEL 57

4.2.1 Internal Model Uncertainty 58

4.2.2 External Uncertainty Impacting the System 58

4.3 POSSIBILITY TO ESTIMATE THE STATE VARIABLE BY MEASUREMENT OF EXPERIMENTAL VARIABLES 59

4.3.1 Uncertainty in the Values Read of the Measured Variable 59

CHAPTER 5 ■ General Scenario Addressed by Kalman Filtering and Specific Cases 61

5.1 ANALYTICAL REPRESENTATION OF A GENERIC KALMAN FILTERING SITUATION 62

5.2 UNIVARIATE ELECTRICAL CIRCUIT EXAMPLE IN THE GENERIC FRAMEWORK 67

5.3 AN INTUITIVE, MULTIVARIATE SCENARIO WITH ACTUAL DYNAMICS: THE FALLING WAD OF PAPER 70

CHAPTER 6 ■ Arriving at the Kalman Filter Algorithm 75

6.1 GOALS AND ENVIRONMENT FOR EACH ITERATION OF THE KALMAN FILTERING ALGORITHM 75

6.2 THE PREDICTION PHASE 76
6.3 MEASUREMENTS PROVIDE A SECOND SOURCE OF KNOWLEDGE FOR STATE ESTIMATION 78
6.4 ENRICHING THE ESTIMATE THROUGH BAYESIAN ESTIMATION IN THE "CORRECTION PHASE" 79

CHAPTER 7 ■ Reflecting on the Meaning and Evolution of the Entities in the Kalman Filter Algorithm 87

7.1 SO, WHAT IS THE KALMAN FILTER EXPECTED TO ACHIEVE? 87
7.2 EACH ITERATION OF THE KALMAN FILTER SPANS "TWO TIMES" AND "TWO SPACES" 88
7.3 YET, IN PRACTICE ALL THE COMPUTATIONS ARE PERFORMED IN A SINGLE, "CURRENT" ITERATION—CLARIFICATION 90
7.4 MODEL OR MEASUREMENT? K_G DECIDES WHO WE SHOULD TRUST 91

PART III **Examples in MATLAB®**

CHAPTER 8 ■ MATLAB® Function to Implement and Exemplify the Kalman Filter 103

8.1 DATA AND COMPUTATIONS NEEDED FOR THE IMPLEMENTATION OF ONE ITERATION OF THE KALMAN FILTER 103
8.2 A BLOCK DIAGRAM AND A MATLAB® FUNCTION FOR IMPLEMENTATION OF ONE KALMAN FILTER ITERATION 106
8.3 RECURSIVE EXECUTION OF THE KALMAN FILTER ALGORITHM 108
8.4 THE KALMAN FILTER ESTIMATOR AS A "FILTER" 110

CHAPTER 9 ■ Univariate Example of Kalman Filter in MATLAB® 113

9.1 IDENTIFICATION OF THE KALMAN FILTER VARIABLES AND PARAMETERS 113
9.2 STRUCTURE OF OUR MATLAB® SIMULATIONS 114

9.3 CREATION OF SIMULATED SIGNALS: CORRESPONDENCE OF PARAMETERS AND SIGNAL CHARACTERISTICS 116
9.4 THE TIMING LOOP 119
9.5 EXECUTING THE SIMULATION AND INTERPRETATION OF THE RESULTS 121
9.6 ISOLATING THE PERFORMANCE OF THE MODEL (BY NULLIFYING THE KALMAN GAIN) 125

CHAPTER 10 ■ Multivariate Example of Kalman Filter in MATLAB® 131

10.1 OVERVIEW OF THE SCENARIO AND SETUP OF THE KALMAN FILTER 131
10.2 STRUCTURE OF THE MATLAB® SIMULATION FOR THIS CASE 134
10.3 TESTING THE SIMULATION 140
10.4 FURTHER ANALYSIS OF THE SIMULATION RESULTS 144
10.5 ISOLATING THE EFFECT OF THE MODEL 148

PART IV **Kalman Filtering Application to IMUs**

CHAPTER 11 ■ Kalman Filtering Applied to 2-Axis Attitude Estimation from Real IMU Signals 153

11.1 ADAPTING THE KALMAN FILTER FRAMEWORK TO ATTITUDE ESTIMATION FROM IMU SIGNALS 154
11.2 REVIEW OF ESSENTIAL ATTITUDE CONCEPTS: FRAMES OF REFERENCE, EULER ANGLES AND QUATERNIONS 154
11.3 CAN THE SIGNALS FROM A GYROSCOPE BE USED TO INDICATE THE CURRENT ATTITUDE OF THE IMU? 159
11.4 CAN WE OBTAIN "MEASUREMENTS" OF ATTITUDE WITH THE ACCELEROMETERS? 160
11.5 SUMMARY OF THE KALMAN FILTER IMPLEMENTATION FOR ATTITUDE ESTIMATION WITH AN IMU 165

11.6 STRUCTURE OF THE MATLAB® IMPLEMENTATION OF THIS KALMAN FILTER APPLICATION 166
11.7 TESTING THE IMPLEMENTATION OF KALMAN FILTER FROM PRE-RECORDED IMU SIGNALS 170

CHAPTER 12 ■ Real-Time Kalman Filtering Application to Attitude Estimation from IMU Signals 179

12.1 PLATFORM AND ORGANIZATION OF THE REAL-TIME KALMAN FILTER IMPLEMENTATION FOR ATTITUDE ESTIMATION 180
12.2 SCOPE OF THE IMPLEMENTATION AND ASSUMPTIONS 181
12.3 INITIALIZATION AND ASSIGNMENT OF PARAMETERS FOR THE EXECUTION 184
12.4 BUILDING (COMPILING AND LINKING) THE EXECUTABLE PROGRAM RTATT2IMU.EXE—REQUIRED FILES 185
12.5 COMMENTS ON THE CUSTOM MATRIX AND VECTOR MANIPULATION FUNCTIONS 186
12.6 INPUTS AND OUTPUTS OF THE REAL-TIME IMPLEMENTATION 189
12.7 TRYING THE REAL-TIME IMPLEMENTATION OF THE KALMAN FILTER FOR ATTITUDE ESTIMATION 191
12.8 VISUALIZING THE RESULTS OF THE REAL-TIME PROGRAM 192

APPENDIX A LISTINGS OF THE FILES FOR REAL-TIME IMPLEMENTATION OF THE KALMAN FILTER FOR ATTITUDE ESTIMATION WITH ROTATIONS IN 2 AXES, 197

REFERENCES, 217

INDEX, 221

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Acknowledgments

The interest in the Kalman Filter that triggered the review of various sources describing it; the reflections about its components; its simulation and real-time implementation was prompted by the participation of the authors in research projects where this kind of estimator can be used advantageously. In particular, the authors wish to acknowledge the support from grants CNS-1532061, IIS-1948254, CNS-1551221, CNS-1338922, BCS-1928502 and CNS-1920182, from the National Science Foundation.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

Authors

Armando Barreto, PhD, is Professor of the Electrical and Computer Engineering Department at Florida International University, Miami, as well as the Director of FIU's Digital Signal Processing Laboratory, with more than 25 years of experience teaching DSP to undergraduate and graduate students. He earned his PhD in electrical engineering from the University of Florida, Gainesville. His work has focused on applying DSP techniques to the facilitation of human–computer interactions, particularly for the benefit of individuals with disabilities. He has developed human–computer interfaces based on the processing of signals and has developed a system that adds spatialized sounds to the icons in a computer interface to facilitate access by individuals with "low vision." With his research team, he has explored the use of Magnetic, Angular-Rate and Gravity (MARG) sensor modules and Inertial Measurement Units (IMUs) for human-computer interaction applications. He is a senior member of the Institute of Electrical and Electronics Engineers (IEEE) and the Association for Computing Machinery (ACM).

Malek Adjouadi, PhD, is Ware Professor with the Department of Electrical and Computer Engineering at Florida International University, Miami. He received his PhD from the Electrical Engineering Department at the University of Florida, Gainesville. He is the Founding Director of the Center for Advanced Technology and Education funded by the National Science Foundation. His earlier work on computer vision to help persons with blindness led to his testimony to the U.S. Senate on the committee of Veterans Affairs on the subject of technology to help persons with disabilities. His research interests are in imaging, signal processing and machine learning, with applications in brain research and assistive technology.

Francisco R. Ortega, PhD, is an Assistant Professor at Colorado State University and Director of the Natural User Interaction Lab (NUILAB). Dr. Ortega earned his PhD in Computer Science (CS) in the field of Human-Computer Interaction (HCI) and 3D User Interfaces (3DUI) from Florida International University (FIU). He also held a position of Post-Doc and Visiting Assistant Professor at FIU. His main research area focuses on improving user interaction in 3DUI by (a) eliciting (hand and full-body) gesture and multimodal interactions, (b) developing techniques for multimodal interaction, and (c) developing interactive multimodal recognition systems. His secondary research aims to discover how to increase interest for CS in non-CS entry-level college students via virtual and augmented reality games. His research has resulted in multiple peer-reviewed publications in venues such as ACM ISS, ACM SUI, and IEEE 3DUI, among others. He is the first-author of *Interaction Design for 3D User Interfaces: The World of Modern Input Devices for Research, Applications and Game Development* (CRC Press, 2020).

Nonnarit O-larnnithipong, PhD, is an Instructor at Florida International University. Dr. O-larnnithipong earned his PhD in Electrical Engineering, majoring in Digital Signal Processing from Florida International University (FIU). He also held a position of Post-Doctoral Associate at FIU between January and April 2019. His research has focused on (1) implementing the sensor fusion algorithm to improve orientation measurement using MEMS inertial and magnetic sensors and (2) developing a 3D hand motion tracking system using IMUs and infrared cameras. His research has resulted in multiple peer-reviewed publications in venues such as *HCI-International* and *IEEE Sensors*.

Introduction

The Kalman Filter, envisioned by Dr. Rudolf E. Kalman (1930–2016) provides an effective mechanism to estimate the state of a dynamic system when a model is available to sequentially predict the state and sequential measurements are also available. This is a common kind of situation in the study of many practical dynamical systems, in diverse fields.

The Kalman Filter has had an important impact on advances within the fields related to the navigation of ships, aircraft and spacecraft. In those initial areas of use, the Kalman Filter was almost exclusively used for highly specific applications, by a very small group of highly specialized users. However, in the XXI century its potential application to small robots and miniature unmanned vehicles has broadened the appeal of this powerful estimation approach to a much wider audience. Furthermore, the recent emergence of cheap, miniaturized Inertial Measurement Units (IMUs) or Magnetic, Angular Rate and Gravity (MARG) sensing modules in contemporary applications has magnified the importance of techniques such as Kalman Filtering, capable of combining information from multiple sensors. Of course, all of these contemporary applications work on digitized data, and, as such, are addressed by the *Discrete Kalman Filter*, which is the subject of this book.

This book is our effort to provide that wider audience with a presentation of the Kalman Filter that is not a mere "cookbook" list of steps (which may result in a sub-optimal use of this important tool), while not requiring the reader to wade through several formal proofs to accomplish a strict derivation of the algorithm. Those proofs were given by Kalman in his 1960 paper and others who have studied the issue from a formal perspective since then.

Instead, it is our hope to provide the reader with an explanation of the Kalman Filter approach that requires a minimum of background concepts, presented simply and succinctly in *Part I* of this book. We expect that this

concise background review will, nonetheless, be enough to help the reader see why it is that the Kalman Filter is capable of obtaining improved estimates of the variables studied. Most importantly, we would like to help the reader acquire an *intuitive understanding* of the meaning of all the parameters involved in the Kalman Filter algorithm and their interactions. The development of that intuitive understanding of the elements at play in the Kalman Filter, and how they come together, according to the background concepts, is the objective of *Part II* of this book. The chapters in Part II lead the reader to the formulation of the Kalman Filter algorithm as the logical conclusion of the considerations made throughout that part of this book. *Part III* of this book focuses on off-line implementation of the Kalman Filter to address estimation challenges of increasing complexity. This third part starts by leveraging the understanding of the Kalman Filter algorithm to develop a MATLAB® implementation for a single Kalman Filter iteration. The rest of the off-line examples in this book are developed from that basic one-iteration function, which can be tailored to create off-line Kalman Filter solutions to diverse estimation problems. *Part IV* of this book includes two chapters that focus on the application of Kalman Filtering to the estimation of orientation of a miniature IMU module, utilizing the gyroscope and accelerometer signals provided by the module. The first of the two chapters of this part develops an off-line solution that still uses the basic one-iteration MATLAB® function created at the beginning of Part III. The gyroscope and accelerometer signals are read from a pre-recorded file which was written while the IMU module was subjected to a specific series of orientation changes, so that the results can be compared with the known series of orientations experienced by the module. The second chapter in Part IV of this book reproduces the implementation of the algorithm in a C program which calculates the Kalman Filter results in real time and stores them to a text file, for later review.

Our emphasis in writing this book has been to foster in the reader an intuitive understanding of the Kalman Filtering process. To that end, we have tried to use analogies and pictorial representations to inform the reader of the main concepts that need to be understood in order to "make sense" of the elements in the Kalman Filter algorithm, their practical meaning, their relationships and the impact that each one of them has in the fulfillment of the goals of the Kalman Filter. Also to that end, we have included a number of "MATLAB® code" segments, which are sometimes scripts (i.e., plain sequences of MATLAB® commands) and some other times custom MATLAB® functions. In either case, the reader

is encouraged to execute the MATLAB® code as originally presented, to observe the results in MATLAB®, *and then explore the concepts by modifying the parameters involved in the code, observing the resulting changes in the outcomes*. This type of exploration of the concepts may be powerful in helping to develop the intuition we want the reader to achieve.

While we, and everyone else, refers to the algorithm that is the subject of this book as a "filter," we would like to warn the reader that the algorithm is really a "state estimator," which, depending on the context, may have multiple signals "coming in" as "inputs" (which we will mainly identify with the "measurements") and yield values of multiple "state variables" after each iteration. Therefore, the Kalman Filter is not constrained to the most familiar framework in which one "input signal" is fed into a "filter" to obtain from it a "cleaner output signal." Nonetheless, some specific configurations of the Kalman Filter estimator may actually involve two signals for which the "cleaning" effect of an input may be apparent in an output signal. We will show a couple of such instances in the examples.

Finally, a few important notes:

- This book focuses on the *Discrete Kalman Filter*. Therefore, with the exception of the initial discussion in Section 1.1, *we will be dealing with discrete-time sequences*. However, to keep compatibility with the notation used in many sources related to Kalman Filtering (including Kalman's 1960 paper) we will denote those sequences as, for example, x(t). In other words, we will use "t" as a *discrete-time* index. (This is in contrast to many signal processing books which use "t" for continuous time and another variable, e.g., "n" to index discrete time.)
- Our discussions will involve scalar variables and matrices (or vectors, as sub-cases of matrices). To differentiate them we will use **bold typeface** for matrices and vectors. For example, $v_1(t)$ represents a *scalar* sequence of (digitized) voltages, while **F**(t) is the state transition *matrix*, which may vary from sampling instant to sampling instant, as it is shown as a function of "t."
- In addition to having included the listings of all the "MATLAB® codes" in the chapter where each is first mentioned, all these files can be retrieved from the repository at https://github.com/NuiLab/IUKF.
- Due to the font and margins used, in some instances, a long line of MATLAB® code may be printed in this book partially "wrapped" into the next line. The files in the repository do not have this effect.

- In the "MATLAB® codes" provided we have used a limited palette of colors for the figures, so that it might still be possible to distinguish several traces in the figures printed using grayscale levels for the physical edition of this book. The readers are invited to change the color assignments to take full advantage of the color capabilities of the computer monitors they will be using when executing the functions in MATLAB®.

MATLAB® is a registered trademark of
The MathWorks, Inc. For product information,
please contact:
The MathWorks, Inc.
3 Apple Hill Drive
Natick, MA 01760-2098 USA
Tel: 508 647 7000
Fax: 508-647-7001
E-mail: info@mathworks.com
Web: www.mathworks.com

I

Background

The objective of the three chapters in this part of the book is to provide the reader with the background concepts that will be essential to understand the elements involved in the kind of estimation challenge that the Kalman Filter addresses. This background is also necessary to follow the reasoning steps that will lead us to the Kalman Filter algorithm, in Part II.

We expect that the reader of this book will be primarily interested in understanding and applying the Kalman Filter. As such, we have tried to keep the background chapters in this part of the book very concise and "to the point." Clearly, any of the concepts outlined in these chapters could be developed much more extensively, and put into a wider, more formal context. Instead we have tried to communicate only the concepts that will be used in our future reasoning (Part II) and we have emphasized descriptions of these concepts in familiar terms, whenever possible.

We strongly suggest that the reader reviews these short background chapters. We hope these chapters will "fill any gaps" in the knowledge of some of our readers, so that they will be well equipped to follow and assimilate our reasoning through Part II. Even if you believe you are fairly familiar with all these background items, it will likely be worthwhile reviewing these chapters to make sure you identify our specific view of these concepts, which should facilitate your reading of Part II.

Taylor & Francis
Taylor & Francis Group
http://taylorandfrancis.com

CHAPTER 1

System Models and Random Variables

This chapter presents the reader with the concept of a model for a system and justifies the need to address some variables as random (and not deterministic) variables. The remainder of this chapter is devoted to providing an intuitive understanding of the typical mechanisms that are used to characterize random variables, emphasizing the understanding of how the characterization applies to the digitized values of a signal. The latter portion of this chapter focuses on the Gaussian (Normal) probability distribution, its notation, the effect of processing samples that have a Gaussian distribution through a transformation represented by a straight line, and the characteristics exhibited by a random variable that is the result of multiplying two variables that have Gaussian probability distributions.

1.1 DETERMINISTIC AND RANDOM MODELS AND VARIABLES

Engineers very often aim at providing real-life solutions to real-life problems. However, engineers very seldom attack the problem directly on the physical reality or circumstance where the "problem" lies. Instead, engineers abstract the critical functional aspects of a real-life problem in an operational model. The model frequently is an intangible representation of the real situation that can, nonetheless, be manipulated to predict what would be the outcomes of different changes in the inputs or the structure of the real-life situation.

Engineers use knowledge of the physical world to emulate real-life constraints in the manipulations of their abstract models, so that results

obtained from the models are good predictors of what will happen if the same manipulations applied to the model are implemented in the real world. Those real-life constraints and the manipulations performed in the model are commonly expressed and achieved in mathematical terms. For example, Kirchhoff's Voltage Law states that the net sum of voltages around a loop is zero (Rizzoni 2004):

$$\sum_i V_i = 0 \tag{1.1}$$

Using this and the model of the relationship between the current i_R through a resistor R and the voltage V_R across its terminals ("Ohm's Law"):

$$V_R = R I_R \tag{1.2}$$

one can calculate, using only algebra, the voltages at the nodes of a resistive circuit with one loop.

For example, if we "simulate" a sinusoidal voltage source, $V_s(t) = \sin(\omega t)$, applied to the circuit in Figure 1.1, we can use the model to develop an "equation" that would predict the value of the output voltage $V_o(t)$ for any time t:

$$V_s(t) - V_{R1}(t) - V_{R2}(t) = 0 \tag{1.3}$$

$$V_s(t) - \left(R1 * I_{R1}(t)\right) - \left(R2 * I_{R2}(t)\right) = 0 \tag{1.4}$$

$$I_{R1}(t) = I_{R2}(t) = I(t) \tag{1.5}$$

$$V_s(t) = (R1 + R2) * I(t) \tag{1.6}$$

$$I(t) = \frac{V_s(t)}{R1 + R2} \tag{1.7}$$

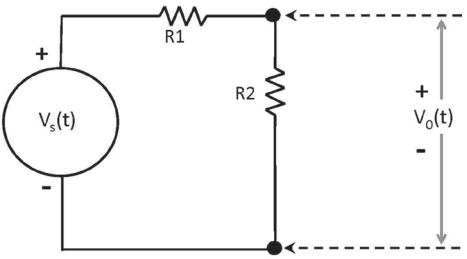

FIGURE 1.1 Simple electrical circuit with just one loop.

And

$$V_o(t) = V_{R2}(t) = R2 * I(t) \tag{1.8}$$

So

$$V_o(t) = V_s(t)\frac{R2}{R1 + R2} \tag{1.9}$$

That is:

$$V_o(t) = \frac{\sin(\omega t) * R2}{(R1 + R2)} \tag{1.10}$$

If we attach to this model "nominal" values of the elements (R1, R2, ω) that replicate the real behavior of the physical components in a real-life circuit, we could "predict" the value of $V_o(t)$ at any desired time t. For example, if R1 = 10,000 Ohm, R2 = 10,000 Ohm and ω = 3.1416 radians per second, then our model predicts:

$$V_o(t) = \frac{\sin(3.1416t) * 10000}{(10000 + 10000)} \tag{1.11}$$

Or

$$V_o(t) = 0.5\sin(3.1416 * t) \quad Volts \tag{1.12}$$

From this we could predict that at t = 0.5 seconds, V_o = 0.5 volts. If t = 1.0 seconds, V_o = 0 V. If t = 1.5 seconds, V_o = –0.5 V, etc.

Equation 1.12 is the analytic expression of a *deterministic* model for $V_o(t)$. You plug in the value of t and the model tells you what the specific value of V_o will be (as estimated by this model). And you get a specific answer for each and any value of t you may want to ask about.

It is important to highlight three aspects of these deterministic models:

1. They will yield a specific value of the output variable for any value of the input variable, "no exceptions," which sounds great, and makes engineers very enthusiastic about them.
2. But, getting "an answer" from these models does not imply in any way that you are getting "the real answer." That is, if the model is not

appropriately matched to the real-life situation, the value of the estimation provided by the model could be completely wrong. (I.e., it could be very different from what happened in the physical system being studied.) As a simple example, consider how wrong the estimates obtained from Equation 1.12 would be if it was used to predict the output voltage of a circuit exactly as described earlier, *but* with a resistor R2 = 1000 Ohms!

3. The deterministic characterization of the signal $V_o(t)$ allows us to fully describe it to another person. I could send an email to a colleague in Germany including Equation 1.12 and she could use it to obtain the same plot of $V_o(t)$ that I can get where I am.

However, we are unable to predict the values of many real-life signals in the definitive way provided by a deterministic characterization, such as Equation 1.12. For example, there is no deterministic model that will accurately tell me the value of voltage coming from a microphone, $V_o(t)$, when I clap my hands next to it.

We know the microphone will yield a voltage that varies through time, alternating above and below 0 V, but there is no equation that can tell me the specific value of voltage that an oscilloscope will show right at t = 0.020 seconds (20 milliseconds), from the moment when my two hands first touch each other when I clap. This is an example of a signal that cannot be easily characterized with a deterministic model. In this example, $V_o(t)$ is best characterized as a "random variable."

This seems to be a considerable setback. How can I characterize a random variable like the voltage signal generated when I clap? Is there anything I can say about it to describe it? Or, in other words, how can I try to describe a random variable? If we were to look at the voltage coming out of a ("dynamic") microphone when I clap, we would notice that there are "certain things" that can be said about it. If we are looking at the voltage straight from the microphone, we will see that it is very unlikely to ever reach outside a (symmetric) range of voltages. For example, it is not likely to be above 1 Volt or below -1 Volt. In fact, it seems that "most of the time" the voltage level is closely above or below 0 Volts.

1.2 HISTOGRAMS AND PROBABILITY FUNCTIONS

Furthermore, there is a "standard way" in which we can summarize these observations about which values are taken by the random signal (particularly well suited for a digitized signal). We can develop the *histogram* of the digitized signal (O'Connell, Orris, and Bowerman 2011; Field 2013).

A histogram for a digitized signal is built simply by dividing a range of amplitudes of the signal in adjacent "bins" of a certain width (for example, one bin from 0.00 to 0.05 and then another bin from 0.05 to 0.10, etc.), and then "placing" each value of the random time series representing the digitized signal in its corresponding bin, according to its amplitude. After doing this for the whole time series, the plot of all the bins, each represented by a rectangle whose height is the number of signal samples ultimately contained in that bin, is the histogram of that signal. Whoever develops the histogram needs to decide first on the location of the bin boundaries. Usually, the width of all the bins is the same. Figure 1.2 shows the creation of a histogram for a random discrete-time signal. (The signal is assumed to have many more than the five samples shown in the figure to illustrate the process.)

Important Note: Please be aware that, from this point forward, we will use "t" as a discrete-time index (McClellan, Schafer, and Yoder 2003, 2016). That is, we will use "t" when referring to discrete samples of a digitized signal, such as x(t). We have chosen to use "t" to represent discrete time to keep compatibility with other texts discussing Kalman Filtering, including Kalman's paper (Kalman 1960).

The histogram is one "practical" tool that we have for the characterization of a random signal. We may not be able to "fully" ("deterministically") forecast the value of the signal at any arbitrary time, but we can at least convey to others which amplitude values the random signal takes on more frequently. In fact, if we normalize the height of each bar in a histogram, dividing it by the total number of signal samples that was used to create it, the resulting graph gives us an empirical sense of "how likely it is that the random signal used to build the histogram will take a value within the range represented by a given bar." For example, if we use 1000 samples of a random signal to develop a histogram and 126 are placed in the histogram bin from 2.0 to 2.1, this will be telling us that 126 / 1000 = 0.126 or 12.6% of this segment of the random variable samples had an amplitude between 2.0 and 2.1. Now, if we were to take another 1000 points *of the same random variable* and develop the same kind of histogram we could provide an educated guess in answering the question: What is the probability that in this new segment a sample will land in the bin from 2.0 to 2.1? We could answer 12.6% (or 0.126). In other words, the normalized histogram provides us with an empirical (coarse) way to determine the probability that a certain random variable (e.g., the amplitude of the random signal) will land in a given interval.

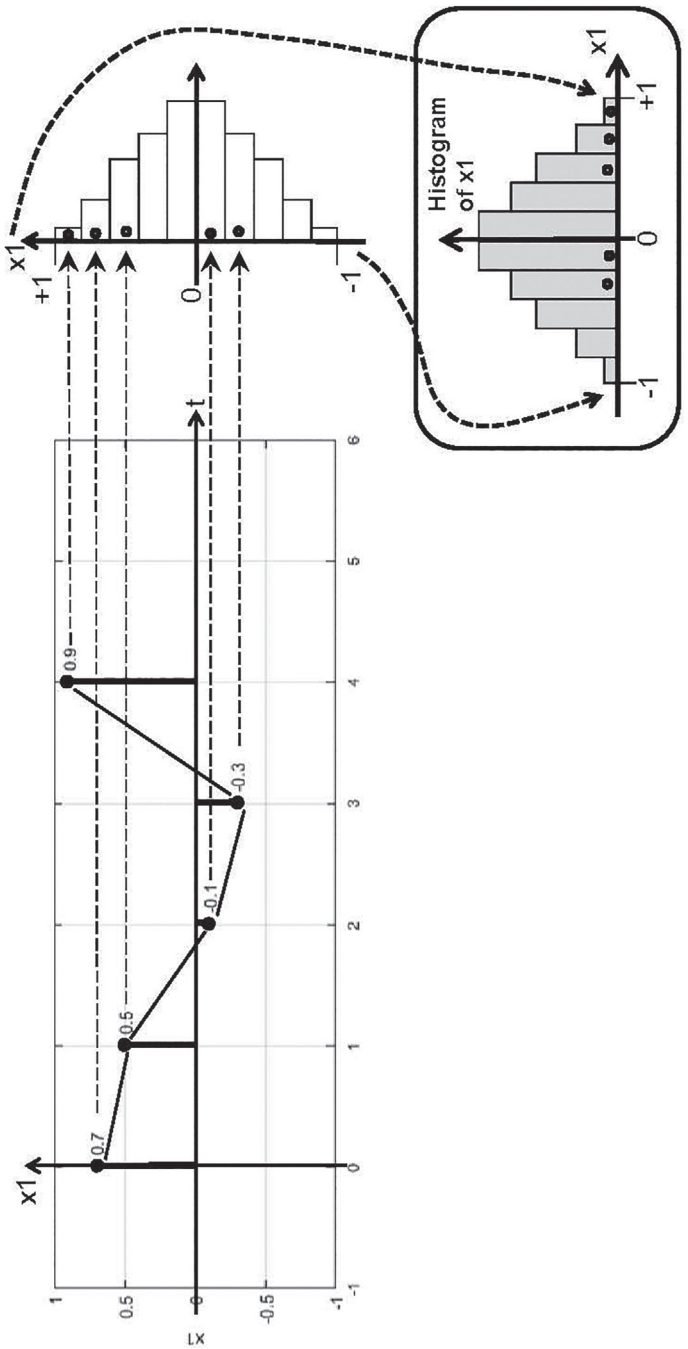

FIGURE 1.2 Creation of the histogram of a random discrete time signal. (Only five samples are shown, as examples.)

As an example, consider the creation of the following random discrete signal in MATLAB® and the development of its normalized histogram (also in MATLAB®). Please note that the MATLAB® command "rng(12345, 'v5normal')" resets the random number generator in MATLAB®, loading the seed value 12345. This will make it possible to obtain repeatable (pseudo) random sequences every time this script is executed. If that is not desired, the line with the rng command can simply be "commented out," by typing a "%" character at the beginning of that line.

```
%%% MATLAB CODE 01.01 ++++++++++++++++++++++++++++++++
rng(12345,'v5normal'); % Resets the Random Number
% Generator to ver5. Normal, seed = 12345;
N = 10000;  % Total number of signal samples
x = randn(N,1);
% NOTE: M' yields the conjugate transpose of matrix M.
% If M is real, M' just transposes it
n = linspace(0,(N-1),N)';
figure; plot(n,x,'k'); grid;
xlabel(' Discrete time index t');
ylabel('x(t)');

% For the Normalized Histogram
% [N x binwidth]/NormFactor = total_area/F = 1.0
% -> NormFactor = N x binwidth
binwidth = 0.1;
EDGES = [-3.0:binwidth:3];
NormFactor = N * binwidth;
figure; bar(EDGES,((histc(x,EDGES))/
NormFactor),0.95,'k');grid;
xlabel('x');
ylabel('Normalized histogram of x');
%%% MATLAB CODE 01.01 ++++++++++++++++++++++++++++++++
```

The normalized histogram shown in Figure 1.4 conveys some interesting information. It tells us that the values of the random digitized signal, or "time series" shown in Figure 1.3, occur with almost equal frequency above and below zero. It also tells us that very few, if any, samples had a value of more than 3 or less than -3. Further, there is a distinct pattern in which the "probability" that a sample value is contained in a given bin decreases from the bins closest to 0 to those close to 3 or -3. It is also clear that the bin that captured the largest number of samples (i.e., the

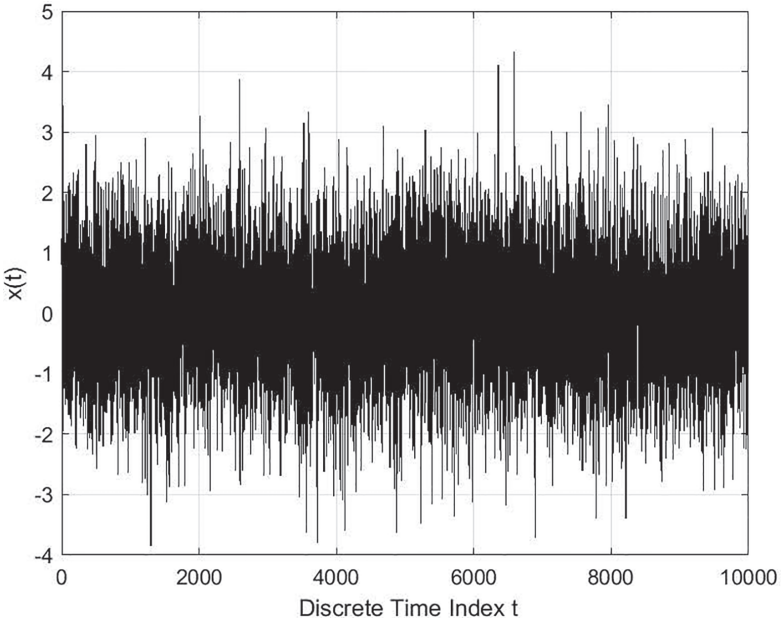

FIGURE 1.3 10,000 points of a random time series with mean 0 and variance 1.0.

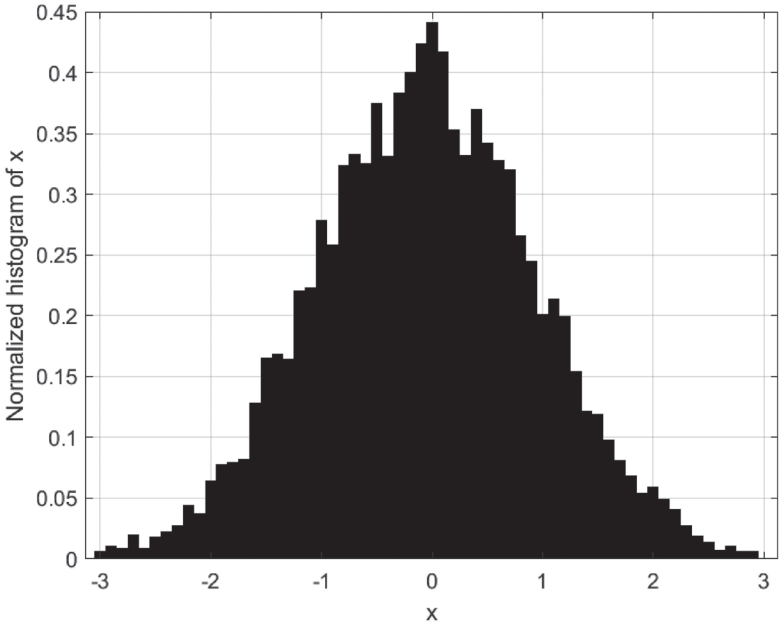

FIGURE 1.4 Normalized histogram of 10,000 points of a random time series with mean 0 and variance 1.0.

"most likely value of the samples") is, in this case, the bin that contains the value of zero. Because of the symmetry of the normalized histogram in Figure 1.4, we can derive that this bin with the highest bar in the normalized histogram is the value that we would obtain if we added all the values of the 10,000 samples processed to create the histogram and divided that sum by 10,000 (i.e., calculating the average). Of course all the values contained in the bin at zero would average to that value (0) and for a given amount of samples in the bar at -1 (for example) there seems to be an approximately equal number of samples captured by the bin at +1. Therefore, the contributions of these two symmetric bars (-1 and +1) to the overall average will not push it above or below the zero value. And this seems to be the case for any pair of bars placed symmetrically to the left and right of that most probable value (0).

A word should be said here about the way in which the normalization was performed in the previous MATLAB® script. Note that we incremented the number of samples for a given bin (for example the one between 2.0 and 2.1), for as long as the amplitude of the discrete-time sequence under consideration fell *anywhere* within that interval. When we observed a sample of amplitude 2.02, we incremented the count for that bin. If we observed a sample of amplitude 2.07, we incremented the count for that bin, etc. Therefore, the total count in a given bin will reflect the probability of finding a value of the random variable throughout the interval that spans what we have called "binwidth" (in the MATLAB® script earlier binwidth = 0.1). Graphically, the probability of that interval is represented as the rectangle for that bin, which has a shaded rectangular area of: base x height = binwidth x bin_count. So, the "total probability" for this random signal (if we placed all or nearly all the samples analyzed into one of the available bins) will be the sum of all the probabilities represented by shaded rectangles, i.e.,

$$\begin{aligned}(binwidth * bin_count1) + \ldots + (binwidth * bin_countLast) \\ = binwidth * TotalCount\end{aligned} \tag{1.13}$$

Again, if all (or nearly all, for an approximation) the samples of the random signal were placed in one of the available bins, the total probability (i.e., the probability that the random variable took on "any" value) must be 1.0 (i.e., certitude). Therefore, using N to represent the total number of samples processed for the histogram: binwidth x N should represent a probability of 1.0.

Since the counts for each of the bins are integer positive numbers (or 0), their total sum will be a positive number bigger than 1. Therefore, a

normalizing factor (NormFactor) must be used to divide all the bin counts in such a way that, after that normalization, the sum of all the rectangular bar areas will be (approximately) 1. From the previous equation, the normalizing denominator needed is:

$$NormFactor = binwidth * N \tag{1.14}$$

This normalization is performed in the MATLAB® script and yields the normalized histogram in Figure 1.4.

1.3 THE GAUSSIAN (NORMAL) DISTRIBUTION

While I could now characterize the random signal for which I drew the normalized histogram by sending a picture of it or the locations and heights of all its bars to my foreign colleague, there is a much more compact mathematical model to encapsulate all this information. The shape of the normalized histogram in Figure 1.4 is found in many cases in nature (just like we find many circles in nature), and a generic mathematical description of this bell shape is available (Mix 1995; Papoulis and Pillai 2002; Yates and Goodman 2014; Hayes 1996):

$$pdf(x) = \frac{1}{\sqrt{2\sigma^2\pi}} e^{\frac{-(x-\mu)^2}{2\sigma^2}} \tag{1.15}$$

This equation yields the "probability density function," pdf, of our random variable, because it can be used to calculate the probability that the random variable x will be contained in a small interval of values of x (much like each one of the bins in the normalized histogram did). This probability density function, in particular, is called a Gaussian pdf, or Gaussian distribution. As mentioned before, this is such a common pdf that it is also known as a "Normal" pdf or a "Normal" distribution (Snider 2017). This is a continuous function of the random variable x. If you evaluate this formula for a large range of values of x, the values of pdf(x) obtained will always describe a bell shape. However, the position of its center and its width will vary, depending on the values of the parameters μ (mean) and σ^2 (variance) used. The mean determines the value of x where the center (axis of symmetry) of the bell shape will be. Therefore, x = μ will have the highest value of pdf(x). In other words, the most likely value of a random variable with this kind of probability density function is the mean, μ. The variance, σ^2, is associated with the "spread" of the different values taken by the random variable x above and below (i.e., higher than or lower than)

the mean. Sometimes it is better to use the square root of the variance: σ, called the standard deviation, to characterize the width of the pdf. In any case, if the values of the random variable x are closely clustered around the mean, the bell shape will be "narrow" and the variance (or standard deviation) will be small in value. If the values of the random variable x are widely dispersed around the mean, the bell shape will be "wide" and the variance (or standard deviation) will be larger in value.

Whether the bell shape is narrow or wide, the area under the curve (from -infinity to +infinity) must always amount to 1.0. This is because of the same reason why the sum of the number of samples captured by all the bars in the histogram must amount to the total number of samples involved in drawing the histogram. (Another way to look at this is to consider that the probability that a sample of the signal could take on a value within the whole range, from -infinity to +infinity, is . . . well 100%!.)

Figure 1.5 shows the result (solid trace) of using Equation 1.15 to find pdf results for values of x from –5 to +5, when the parameters considered were $\mu = 0$ and $\sigma^2 = 1.0$. It should be noted that the profile of this "abstract" pdf model matches very closely the practically determined profile of our normalized histogram.

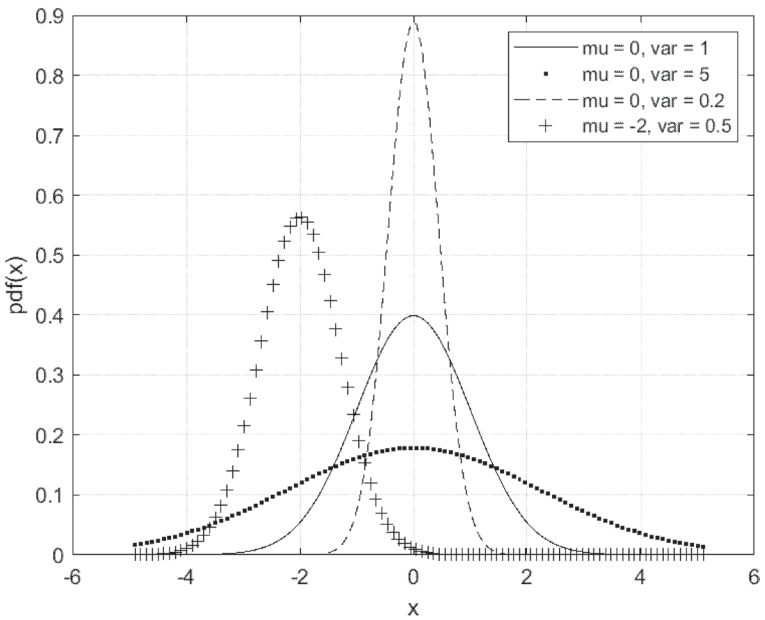

FIGURE 1.5 Gaussian Distributions. The solid trace corresponds to a Gaussian distribution with 0 mean and variance of 1. Three other combinations of mean and variance are also shown.

Because the Gaussian (Normal) distribution and its formula are well known everywhere, I could now characterize my random variable to my foreign friend by saying that I am modeling it as a Gaussian (Normal) probability density function with mean $\mu = 0$ and variance $\sigma^2 = 1.0$. Furthermore, also by convention, I could use well-established notation (McCool 2012) to just say that I am modeling the pdf of my random variable as:

$$\boldsymbol{N}\left(\mu,\sigma^2\right), \textit{which in this case would be } \boldsymbol{N}\left(0.0,1.0\right) \tag{1.16}$$

This is a highly compact form to characterize what we can actually say about the random signal that we are dealing with. With this piece of information, my friend could plug in $\mu = 0$ and $\sigma^2 = 1.0$ in Equation 1.15 and then draw the plot of the bell shape. With that she could estimate how likely it will be to observe the random variable take a value within any range (which would be the area under the pdf curve between the beginning and end values of the range in question). Additionally, the plot (or, for that matter the numerical values of μ and σ^2), will immediately reveal where is the center of the pdf, which is the most likely value of x to be observed, and how widely spread the occurrences will be around that center.

Given how informative the mean and the variance of a Gaussian pdf are, it is very useful to have these equations to calculate them numerically (directly) from a random time series (or "sample") x(t), comprising N values (Johnson and Wichern 2002):

$$\mu_x = \frac{1}{N}\sum_t x\left(t\right) \qquad (\text{“}\textit{sample mean}\text{”}) \tag{1.17}$$

$$\sigma_x^2 = \frac{1}{N}\sum_t \left(x\left(t\right)-\mu_x\right)^2 \qquad (\text{“}\textit{sample variance}\text{”}) \tag{1.18}$$

where t is being used to represent *discrete time*, so that x(t) are the samples contained in a discrete-time series (for example the values obtained from an analog-to-digital converter).

1.4 MODIFICATION OF A SIGNAL WITH GAUSSIAN DISTRIBUTION THROUGH A FUNCTION REPRESENTED BY A STRAIGHT LINE

Another important consideration for our study is to discuss what happens to a Gaussian random variable (in terms of its mean and its variance)

when the samples of the original random time series x(t) are processed by a function represented by a straight line. This is the case of a transformation that just multiplies the input signal samples by a constant coefficient and adds a constant value to the product, following a formula such as this:

$$y(t) = mx(t) + b \tag{1.19}$$

A straight line, with slope m and intersecting the vertical axis at b, can be used to illustrate the projection of every sample of the input time series x(t) to become the corresponding output sample of the resulting time series y(t). So, for example, if a transformation is given by

$$y(t) = 2x(t) + 1.5 \tag{1.20}$$

Figure 1.6 shows how a few samples of the random signal x(t) used in the previous discussion would be projected to y(t).

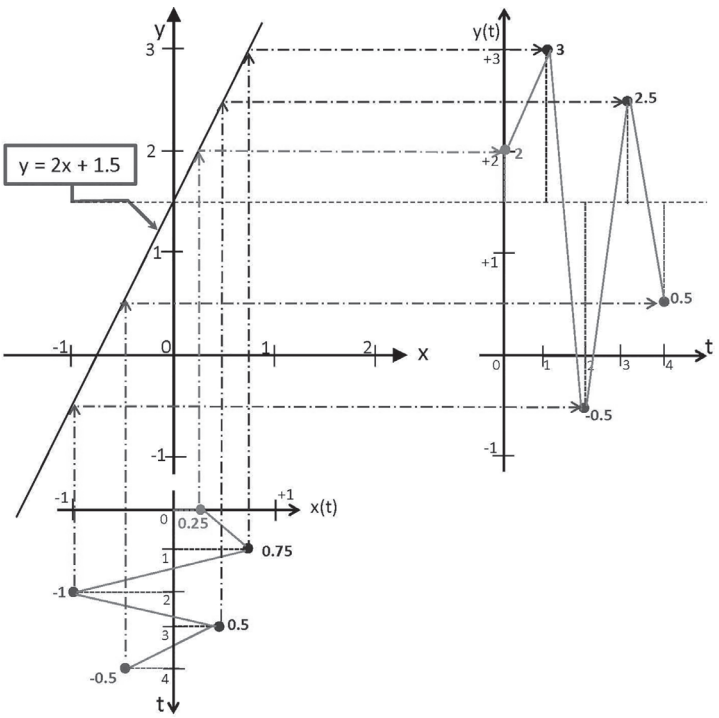

FIGURE 1.6 Projection of some samples of the input sequence x(t) to the output sequence y(t).

Note: A function such as the one indicated by Equation 1.20 cannot be called a "linear transformation" because it does not have the properties of homogeneity and additivity which are required to consider a transformation "linear" (Lathi 1998). For example, for x = 1, y = 3.5; for x = 4, y = 9.5; but, for x = 1 + 4 = 5, y will be 11.5, and *not* 3.5 + 9.5 = 13. The transformation indicated by Equation 1.19 is truly linear if b = 0.

If we project all 10,000 samples of x(t) to obtain the corresponding 10,000 samples of y(t), the comparison of the first 100 samples of both signals is shown in Figure 1.7. The comparison of their normalized histograms is shown in Figure 1.8. These figures were obtained with the following MATLAB® commands:

```
%%% MATLAB CODE 01.02 ++++++++++++++++++++++++++++++++
rng(12345,'v5normal'); % Resets the Random Number
%Generator to ver5. Normal, seed = 12345;

% Generate x
x = randn(10000,1);
n = linspace(0,9999,10000)';

% Obtain y by the transformation y = 2 x + 1.5

y = 2 * x + 1.5;

% Plot first 100 points of both time series (same
%graphical scale)
figure;
subplot(2,1,1);
plot(n(1:100),x(1:100),'k');
axis([0,99,-9, 9]);
grid;
xlabel('Discrete time index t');
ylabel('x(t)')
subplot(2,1,2)
plot(n(1:100),y(1:100),'k');
axis([0,99,-9, 9]);
grid;
xlabel('Discrete time index t');
ylabel('y(t)')

% NORMALIZED HISTOGRAMS
N = 10000;
```

```
EDGES = [-9.0:0.1:9];
binwidth = 0.1;
NormFactor = N * binwidth ;

figure;
subplot(2,1,1);
bar(EDGES,((histc(x,EDGES))/NormFactor),1.02,'k');grid;
axis([-9,9,0, 0.5]);
xlabel('x');
ylabel('Normalized Histogram of x')

subplot(2,1,2);
bar(EDGES,((histc(y,EDGES))/NormFactor),1.02,'k');grid;
axis([-9,9,0, 0.5]);
xlabel('y');
ylabel('Normalized Histogram of y')
%%% MATLAB CODE 01.02 ++++++++++++++++++++++++++++++++
```

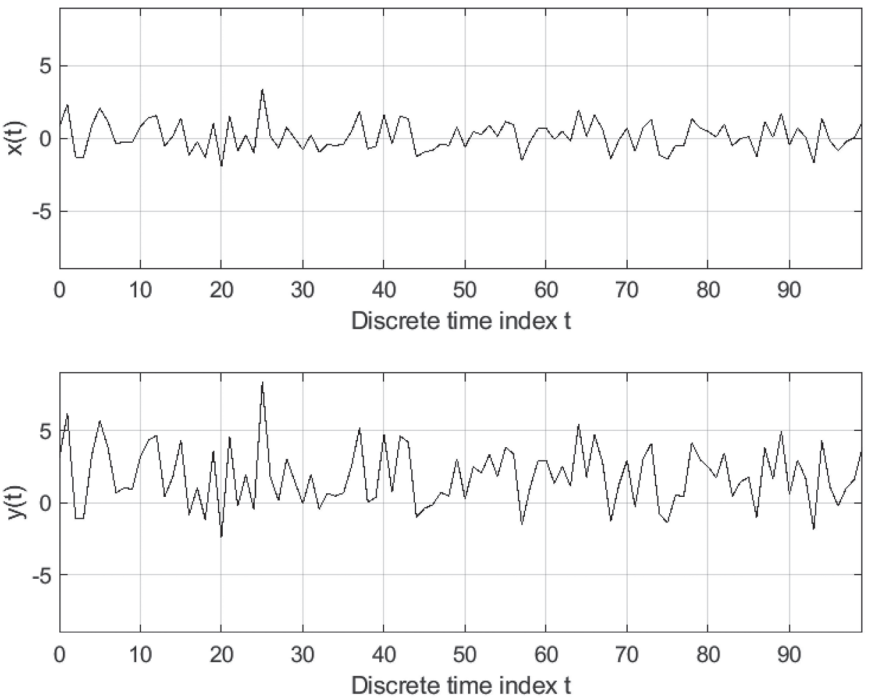

FIGURE 1.7 First 100 samples of x(t) and first 100 samples of y(t) = 2 x(t) + 1.5.

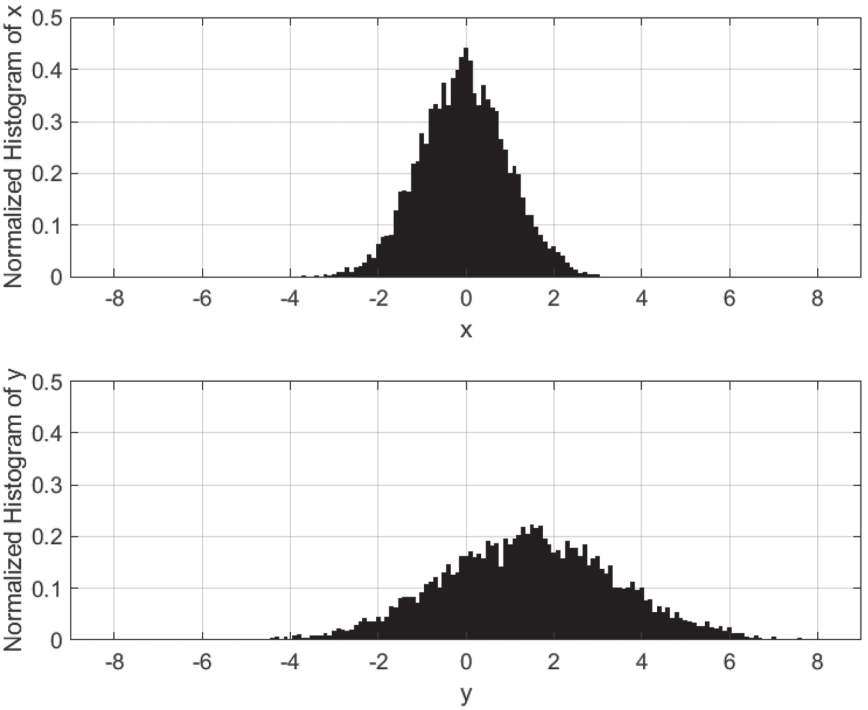

FIGURE 1.8 Normalized histogram of x(t) and normalized histogram of y(t) = 2 x(t) + 1.5.

The comparison of the time series confirms our intuition with respect to the distribution of samples for the output signal: We are duplicating the amplitude of the original signals around the mean (0) and therefore the samples of y(t) are more widely spread. Also, we are adding 1.5 to each input sample (after it has been duplicated in amplitude), therefore the whole output time series now fluctuates above and below a value of 1.5. That is, the new mean is 1.5. The comparison of the normalized histograms further confirms our observations: The mean value has been shifted to b = 1.5, and the spread of the y(t) samples is wider than the spread of the x(t) samples around their corresponding means. Moreover, we see that the shape of the pdf for the output samples *is also a bell shape*. (The "height" of the bell has to be smaller, because its total area must still be 1.0.) These observations are not the exception, but the rule. In general, if samples of a Gaussian distribution $\mathcal{N}(\mu_x,\sigma^2_x)$ are processed by a transformation y = m

x + b, then the output random variable will have a Gaussian distribution with mean

$$\mu_y = m\mu_x + b \tag{1.21}$$

and variance

$$\sigma_y^2 = m^2\sigma_x^2 \tag{1.22}$$

In other words, the output of the transformation y = m x + b will have a Gaussian pdf:

$$\boldsymbol{N}\left(\mu_y, \sigma_y^2\right) = \boldsymbol{N}\left(m\mu_x + b, \sigma_x^2 * m^2\right) \tag{1.23}$$

Putting it into words, we would say that the mean of the resulting Gaussian is obtained by simply "processing" the mean of the input Gaussian. And the variance of the resulting Gaussian is obtained by multiplying the variance of the input Gaussian times m^2 (McCool 2012). It should be noticed then that the output Gaussian could be wider than the input Gaussian if m > 1 (as in our example) or it could be narrower, if m < 1. It is also apparent that the b constant in the transformation only has an impact on the mean of the output distribution, but does not affect the variance of the output distribution.

In the example

$$\mu_y = m\mu_x + b = (2)(0) + 1.5 = 1.5 \tag{1.24}$$

and

$$\sigma_y^2 = m^2\sigma_x^2 = \left(2^2\right)(1.0) = 4.0 \tag{1.25}$$

This is apparent in Figure 1.8 by the shifting and widening of the bell shape of the output histogram, in comparison to the bell shape of the input histogram.

One graphical way to represent these changes of the pdf through a transformation represented by a line is as in Figure 1.9.

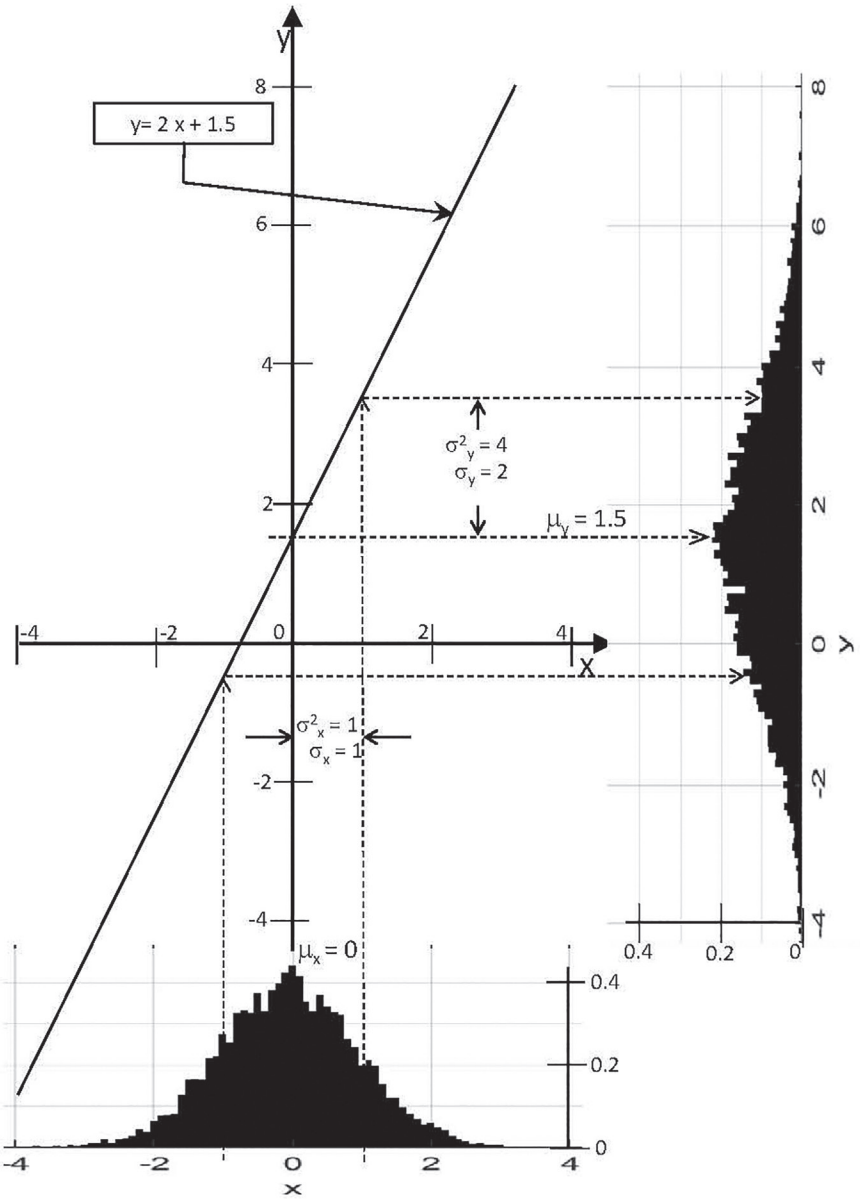

FIGURE 1.9 Effects of projecting a random variable, x(t) through the transformation y(t) = 2 x(t) + 1.5.

1.5 EFFECTS OF MULTIPLYING TWO GAUSSIAN DISTRIBUTIONS

Consider the product of two Gaussian distributions $\mathcal{N}(\mu_1, \sigma^2_1)$ and $\mathcal{N}(\mu_2, \sigma^2_2)$. Then, it is known that the mean and variance of the resulting Gaussian distribution $\mathcal{N}(\mu_p, \sigma^2_p)$ are (Bromiley 2003):

$$\mu_P = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2} = \mu_2 + \frac{\sigma_2^2\left(\mu_1 - \mu_2\right)}{\sigma_1^2 + \sigma_2^2} \tag{1.26}$$

and

$$\sigma_p^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2} = \sigma_2^2 - \frac{\sigma_2^4}{\sigma_1^2 + \sigma_2^2} \tag{1.27}$$

(These two equations first present the most common formats for these results, and then they also present alternative formats, more appropriate for our discussions, which can be obtained after a few simple algebraic manipulations.)

And, if we define the parameter k as this ratio of variances:

$$k = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} \tag{1.28}$$

Then

$$\mu_P = \mu_2 + k\left(\mu_1 - \mu_2\right) \tag{1.29}$$

and

$$\sigma_p^2 = \sigma_2^2 - k\sigma_2^2 = \sigma_2^2\left(1 - k\right) \tag{1.30}$$

As an example, consider the product of these two Gaussian distributions: $\mathcal{N}_1(1, 1)$ and $\mathcal{N}_2(6, 0.25)$. First, using the original equations (Equations 1.26 and 1.27):

$$\mu_P = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2} = \frac{(1)(0.25) + (6)(1)}{(1) + (0.25)} = 5 \tag{1.31}$$

$$\sigma_p^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2} = \frac{(1)(0.25)}{(1) + (0.25)} = 0.2 \tag{1.32}$$

And then with the equations that use the parameter k:

$$k = \frac{(0.25)}{(1)+(0.25)} = 0.2 \tag{1.33}$$

$$\mu_p = 6 + (0.2)(1-6) = 5 \tag{1.34}$$

$$\sigma_p^2 = (0.25)(1-(0.2)) = 0.2 \tag{1.35}$$

So, we verify that the original formulas, as well as the modified formulas (with the parameter k) yield the same results for the mean and the variance of the product.

We would like to call the reader's attention to an important and interesting observation. The characteristics of the resulting (product) Gaussian are influenced more by the original Gaussian that had a smaller variance. In our example $\sigma^2_2 = 0.25$ is smaller than $\sigma^2_1 = 1$. Notice how, indeed, $\mu_p = 5$ ended up being much closer to $\mu_2 = 6$ than to $\mu_1 = 1$. Also, the resulting variance, $\sigma^2_p = 0.2$ is closer to $\sigma^2_2 = 0.25$ than it is to $\sigma^2_1 = 1$. It is interesting that, actually $\sigma^2_p < \sigma^2_2$. Figure 1.10 helps to visualize these observations.

While we have referred in this last section to the product of two "Gaussian distributions," it should be understood that the conclusions reached are applicable when we multiply samples of data series

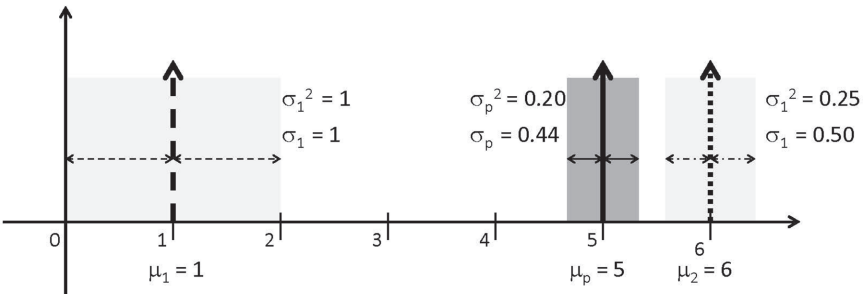

FIGURE 1.10 When a first Gaussian distribution (mean = 1, variance = 1) is multiplied with a second Gaussian distribution (mean = 6.0, variance = 0.25) the resulting Gaussian function has a mean (5.0) and a variance (0.2) that are closer to the mean and the variance of the original Gaussian with the lowest variance. The horizontal arrows span 1 standard deviation, in each case.

that have those Normal distributions. That is, if we multiply samples from a time series $x_1(t)$ whose probability function is $\mathcal{N}(\mu_1, \sigma^2_1)$ with samples of a second time series $x_2(t)$, whose probability function is $\mathcal{N}(\mu_2, \sigma^2_2)$, the resulting time series, $x_p(t)$, will have a Gaussian probability function $\mathcal{N}(\mu_p, \sigma^2_p)$, with μ_p and σ^2_p determined according to Equations 1.31 to 1.35.

REFERENCES

Aggarwal, P., Z. Syed, and A. Noureldin. 2010. *MEMS-Based Integrated Navigation*: Artech House, Boston, MA.

Ahrendt, Peter. 2005. "The Multivariate Gaussian Probability Distribution." Technical University of Denmark, Technical Report.

Anderson, B.D.O., and J.B. Moore. 2012. *Optimal Filtering*: Dover Publications, Mineola, NY.

Ayman, S. 2016. *Sensor Fusion Using Kalman Filter for a Quadrotor-Attitude Estimation: Basics, Concepts, Modelling, MATLAB Code and Experimental Validation*: LAP Lambert Academic Publishing, Saarbrucken, Germany.

Bishop, Gary, and Greg Welch. 2001. "An Introduction to the Kalman Filter." *Proceedings of SIGGRAPH, Course* 8 (27599–23175): 41.

Bromiley, P. A. 2003. "Products and Convolutions of Gaussian Distributions." Medical School, University of Manchester, Manchester, Technical Report 3.

Brown, R.G., and P.Y.C. Hwang. 2012. *Introduction to Random Signals and Applied Kalman Filtering with MATLAB Exercises, 4th Edition*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Candy, J.V. 2009. *Bayesian Signal Processing: Classical, Modern and Particle Filtering Methods*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Catlin, D.E. 1988. *Estimation, Control, and the Discrete Kalman Filter*: Springer, New York, NY.

Childers, D.G. 1997. *Probability and Random Processes: Using MATLAB with Applications to Continuous and Discrete Time Systems*: Irwin (McGraw-Hill), New York, NY.

Cox, B., and A. Cohen. 2014. *Human Universe*: William Collins, (Harper Collins), New York, NY.

Drake, S. 1978. *Galileo at Work: His Scientific Biography*: University of Chicago Press, Chicago, IL.

Farrell, J.A. 2008. *Aided Navigation: GPS with High Rate Sensors*: McGraw-Hill Higher Education, New York, NY.

Field, A. 2013. *Discovering Statistics Using IBM SPSS Statistics*: Sage Publications, London, England.

Gales, M.J.F., and S.S. Airey. 2006. "Product of Gaussians for Speech Recognition." *Computer Speech & Language* 20 (1): 22–40. doi:10.1016/j.csl.2004.12.002.

Gelb, A., J.F. Kasper, Analytic Sciences Corporation, Technical Staff, and Analytic Sciences Corporation. 1974. *Applied Optimal Estimation*: M.I.T. Press, Cambridge, MA.

Grewal, M.S., and A.P. Andrews. 2008. *Kalman Filtering: Theory and Practice Using MATLAB*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Haug, A.J. 2012. *Bayesian Estimation and Tracking: A Practical Guide*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Hayes, M.H. 1996. *Statistical Digital Signal Processing and Modeling*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Holman, J.P. 2001. *Experimental Methods for Engineers*: McGraw-Hill Higher Education, New York, NY.

Johnson, R.A., and D.W. Wichern. 2002. *Applied Multivariate Statistical Analysis*: Prentice Hall, Upper Saddle River, NJ.

Kalman, R.E. 1960. "A New Approach to Linear Filtering and Prediction Problems." *Journal of Basic Engineering* 82 (1): 35–45. doi:10.1115/1.3662552.

Kovvali, N., M. Banavar, and A. Spanias. 2013. *An Introduction to Kalman Filtering with MATLAB Examples*: Morgan & Claypool Publishers, San Rafael, CA.

Kuipers, J.B. 2002. *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*: Princeton University Press, Princeton, NJ.

Lathi, B.P. 1998. *Signal Processing and Linear Systems*: Berkeley-Cambridge Press, Carmichael, CA.

Maybeck, P.S. 1979. *Stochastic Models, Estimation and Control*: Academic Press, New York, NY.

McClellan, J.H., R.W. Schafer, and M.A. Yoder. 2003. *Signal Processing First*: Pearson Education International, Upper Saddle River, NJ.

McClellan, J.H., R.W. Schafer, and M.A. Yoder. 2016. *DSP First*: Pearson, Upper Saddle River, NJ.

McCool, J.I. 2012. *Using the Weibull Distribution: Reliability, Modeling, and Inference*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Mix, D.F. 1995. *Random Signal Processing*: Prentice Hall, Englewood Cliffs, NJ.

O'Connell, R., J.B. Orris, and B. Bowerman. 2011. *Essentials of Business Statistics*: McGraw-Hill Higher Education, New York, NY.

Papoulis, A., and S.U. Pillai. 2002. *Probability, Random Variables, and Stochastic Processes*: McGraw-Hill Higher Education, New York, NY.

Peebles, P.Z. 2001. *Probability, Random Variables, and Random Signal Principles*: McGraw-Hill Higher Education, New York, NY.

Rizzoni, G. 2004. *Principles and Applications of Electrical Engineering*: McGraw-Hill Higher Education, New York, NY.

Serra, G.L. 2018. *Kalman Filters: Theory for Advanced Applications*: IntechOpen, Rijeka, Croatia.

Shoemake, Ken. 1985. "Animating Rotation with Quaternion Curves." *ACM SIGGRAPH Computer Graphics* 19 (3): 245–254

Simon, Dan. 2001. "Kalman Filtering." *Embedded Systems Programming* 14 (6): 72–79.

Snider, A.D. 2017. *Random Processes for Engineers: A Primer*: CRC Press, Taylor & Francis Group, Boca Raton, FL.

Spagnolini, U. 2018. *Statistical Signal Processing in Engineering*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Stark, H., and J.W. Woods. 2002. *Probability and Random Processes with Applications to Signal Processing*: Prentice Hall, Englewood Cliffs, NJ.

Stengel, R.F. 1994. *Optimal Control and Estimation*: Dover Publications, New York, NY.

Stone, J.V. 2013. *Bayes' Rule: A Tutorial Introduction to Bayesian Analysis*: Sebtel Press, Sheffield.

Stone, J.V. 2015. *Bayes' Rule with MATLAB: A Tutorial Introduction to Bayesian Analysis*: Sebtel Press, Sheffield.

Stone, J.V. 2016a. *Bayes' Rule with Python: A Tutorial Introduction to Bayesian Analysis*: Sebtel Press, Sheffield.

Stone, J.V. 2016b. *Bayes' Rule with R: A Tutorial Introduction to Bayesian Analysis*: Sebtel Press, Sheffield.

Stone, J. V. 2019. "An Introduction to Bayes Rule (Chapter 1, Online)." [Web Page], accessed 11/28/2019. http://jim-stone.staff.shef.ac.uk/BookBayes2012/HTML_BayesRulev5E-bookHTMLFiles/ops/xhtml/ch01BayesJVSone.html.

Titterton, D.H., and J.L. Weston. 2004. *Strapdown Inertial Navigation Technology*: American Institute of Aeronautics and Astronautics, Reston, VA.

Wertz, J.R. 1980. *Spacecraft Attitude Determination and Control*: Reidel, Dordrecht, Holland.

Yates, R.D., and D.J. Goodman. 2014. *Probability and Stochastic Processes: A Friendly Introduction for Electrical and Computer Engineers*: John Wiley & Sons, Incorporated, Hoboken, NJ.

Yun, X., E. R. Bachmann, and R. B. McGhee. 2008. "A Simplified Quaternion-Based Algorithm for Orientation Estimation from Earth Gravity and Magnetic Field Measurements." *IEEE Transactions on Instrumentation and Measurement* 57 (3): 638–650. doi:10.1109/tim.2007.911646.

Zarchan, P., H. Musoff, F.K. Lu, American Institute of Aeronautics, and Astronautics. 2009. *Fundamentals of Kalman Filtering: A Practical Approach*: American Institute of Aeronautics and Astronautics, Reston, VA.